PYTHON

Python is a high-level, interpreted programming language used to write instructions for computers in a simple and readable way.

Python is a programming language that is commonly used to create websites and applications, automate processes, and do data analysis.

Why Python is popular

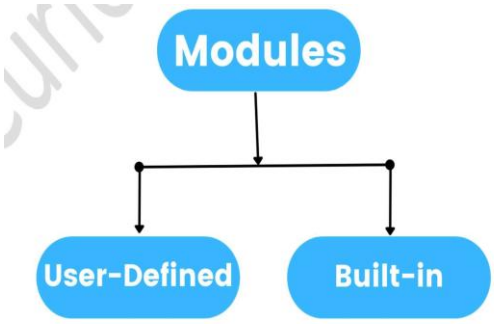
- Easy to learn (simple syntax)
- Works on all platforms (Windows, Mac, Linux)
- Huge community and libraries
- Used in many fields

What can be done using python?

- **Python** is a versatile programming language used for web development, software development, automation, data analysis, and data visualization. Its simple syntax makes it popular among both programmers and non-programmers like accountants and scientists.
- In **data analysis and machine learning**, Python helps perform complex calculations, create visualizations, manage data, and build machine learning models.
- In **web development**, Python is mainly used for backend work such as handling servers, databases, and application logic. Popular frameworks include Django and Flask.
- **Automation or Scripting in Python** refers to using code to perform repetitive tasks automatically. It helps save time and increase efficiency by handling tasks like file processing, error checking, simple calculations, and removing duplicate data.

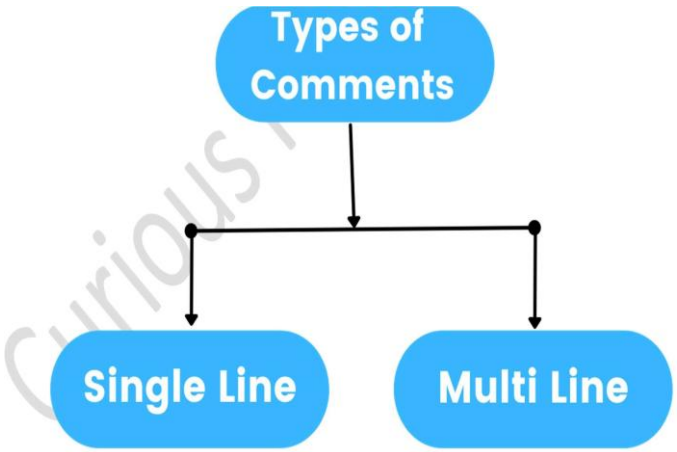
MODULES IN PYTHON

- A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables.
- A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.



COMMENTS IN PYTHON

- Python Comment is an essential tool for the programmers. Comments are generally used to explain the code.
- We can easily understand the code if it has a proper explanation.
- A good programmer must use the comments because in the future anyone wants to modify the code as well as implement the new module; then, it can be done easily.



SINGLE LINE COMMENT

- Python single line comment starts with the hashtag
- symbol (#) with no white spaces and lasts till the end of the line.
- If the comment exceeds one line then put a hashtag on the next line and continue the comment

PIP

- pip is a package-management system written in Python used to install and manage software packages.
- It connects to an online repository of public packages, called the Python Package Index.
- pip can also be configured to connect to other package repositories, provided that they comply to Python Enhancement Proposal 503.

VARIABLES, DATATYPES AND KEYWORDS

- A **variable in Python** is a name used to store a value in memory.
- Python automatically determines the variable's type, so no declaration is needed.
- Python automatically determines the variable's type, so no declaration is needed.
  Variable names must start with a letter or underscore and can include letters and digits.

  Examble:
  x = 10 # integer
  name = "John" # string
  price = 99.5 # float

  IDENTIFIER IN PYTHON

  Variables are examples of identifiers, which are names used to identify data in a program.
  Rules:
- Must start with a **letter or underscore (_)**
- Can contain **letters, digits, and underscores**

- Cannot contain **spaces or special characters** (like !, @, #, etc.)
- Identifier name must not be similarto any keyword defined in the language.
- Examples of valid identifiers: a123, _n, n_9, etc.
- Examples of invalid identifiers: 1a, n%4, n 9, etc.

DATA TYPES IN PYTHON

- Variables store values, and each value has a data type. Python is a dynamically typed language, so we do not need to declare the type of a variable. The interpreter automatically assigns the data type based on the value.
- The variable `a` stores the value 5, and we do not need to specify its type. Python automatically treats it as an integer.
- We can also check the type of a variable using the `type()` function, which returns the data type of the given value.

PYTHON DATA TYPES

1)Numbers: Number stores numeric values. The integer, float, and complex values belong to a Python Numbers data-type.

- Python supports three types of numeric data-
- Integer - Integer value can be any length such as integers 10, 2, 29, -20, -150 etc.
- Float- Float is used to store floating-point numbers like
    - 1.9, 9.902, 15.2, etc.

2) Sequence Type:

- String: The string can be defined as the sequence of characters represented in the quotation marks. In Python, we can use single, double, or triple quotes to define a string.
- List: Python Lists are similar to arrays in C. However, the list can contain data of different types. The items stored in the list are separated with a comma (,) and enclosed within square brackets [].
    - Tuple: A tuple is similar to the list in many ways. Like lists, tuples also contain the collection of the items of different data types. The items of the tuple are separated with a comma (,) and enclosed in parentheses ().

3) Boolean Data Type in Python

- The Boolean data type represents only two values: True and False. It is used to check conditions and make decisions in a program.

4) Set in Python

- A set is a collection data type that stores unordered and unique elements. It does not allow duplicate values.
- Unordered collection
- No duplicate values allowed
- Mutable (can be changed)
- Written using curly brackets {}

5) Dictionary in Python

- A dictionary is a collection data type that stores data in key-value pairs. Each key is unique and is used to access its corresponding value.

- Stores data in key : value format
- Keys must be unique
- Values can be of any data type
- It is mutable (can be changed)
- Written using curly brackets { }

OPERATOR IN PYTHON

The operator can be defined as a symbol which is responsible for a particular operation between two operands. Python

provides variety of operators, which are described as follows.

- **Arithmetic operators**
- **Comparison operators**
- **Assignment Operators**
- **Logical Operators**
- **Membership Operators**
- **Identity Operators**

1. Arithmetic Operators

These operators are used for mathematical operations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
**	Exponent (power)	a ** b

Example:

```
a = 10
b = 3
print(a + b)  # 13
print(a % b)  # 1
```

---

2. Comparison Operators

These operators compare two values and return True or False.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

```
a = 10
b = 5
print(a > b)   # True
```

```
print(a == b)  # False
```

---

3. Logical Operators

Used to combine conditions.

Operator	Meaning
and	True if both conditions are true
or	True if any one condition is true
not	Reverses the result

Example:

```
x = True
y = False
print(x and y)  # False
print(x or y)   # True
```

---

4. Assignment Operators

Used to assign values to variables.

Operator	Meaning
=	Assign
+=	Add and assign
-=	Subtract and assign

Example:

```
x = 10
x += 5   # x = x + 5
print(x)  # 15
```

---

5. Logical Operators

Logical operators are used to combine multiple conditions and return True or False.

Operator	Meaning
and	True if both conditions are True
or	True if at least one condition is True
not	Reverses the result

Example:

```
a = True
b = False

print(a and b)  # False
print(a or b)   # True
print(not a)    # False
```

---

6. Membership Operators

Membership operators are used to check whether a value is present in a sequence (like list, string, tuple).

Operator	Meaning
in	Returns True if value is found
not in	Returns True if value is not found

Example:

```
fruits = ["apple", "banana", "mango"]

print("apple" in fruits)    # True
print("grape" not in fruits) # True
```

---

7. Identity Operators

Identity operators are used to check whether two variables refer to the same object in memory.

Operator	Meaning
is	Returns True if both variables point to the same object
is not	Returns True if they are not the same object

Example:

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b)    # True
print(a is c)    # False
```

8.Bitwise Operators

Bitwise operators are used to perform operations on numbers at the bit level (binary form). They work directly on the binary representation of integers.

Common Bitwise Operators

Operator	Name	Meaning
&	AND	Sets bit to 1 if both bits are 1
		OR
^	XOR	Sets bit to 1 if bits are different
~	NOT	Inverts all bits
<<	Left Shift	Shifts bits to left (multiplies by 2)
>>	Right Shift	Shifts bits to right (divides by 2)

INPUT IN PYTHON

Python input() Function:Python input() function is used to get input from the user. It prompts for the user input and reads a line. After reading data, it converts it into a string and returns that. It throws an error EOFError if EOF is read.

Parameters**prompt** : It is a string message which prompts for the user input

Return :It returns user input after converting into a string.

Let's see some examples of input() function to understand it's functionality.

List,Tuples,set,Dictionary and Type Conversion

List in Python

A **list** in Python is a collection data type used to store **multiple values in a single variable**. It is ordered and allows duplicate values.

Key Features of List

- Ordered (elements have a fixed position)
- Mutable (can be changed)
- Allows duplicates
- Can store different data types
- Written using **square brackets [ ]**

Tuples in Python

A **tuple** in Python is a collection data type used to store **multiple values in a single variable**. It is similar to a list, but it is **immutable (cannot be changed)**.

Key Features of Tuple

- Ordered (elements have a fixed position)
- Immutable (cannot be modified after creation)
- Allows duplicate values
- Can store different data types
- Written using **round brackets ( )**

Set in Python

A **set** in Python is a collection data type used to store **multiple items in a single variable**. It is **unordered** and does not allow **duplicate values**.

Key Features of Set

- Unordered (no fixed position of elements)

- Does not allow duplicates
- Mutable (can be changed)
- Written using **curly brackets { }**
- Stores only unique values

Dictionary in Python

A **dictionary** in Python is a collection data type used to store data in **key-value pairs**. Each key is unique and is used to access its corresponding value.

Key Features of Dictionary

- Stores data in **key : value** format
- Keys must be **unique**
- Values can be duplicated
- Mutable (can be changed)
- Written using **curly brackets { }**

Explicit Type Casting

In this method, Python need user involvement to convert the variable data type into certain data type in order to the operation required.

Mainly in type casting can be done with these data type function:

Int() : Int() function take float or string as an argument and return int type object.

float() : float() function take int or string as an argument and return float type object.

str() : str() function take float or int as an argument and return string type object.

FLOW OF CONTROL

Flow of Control in Python
Flow of control refers to the order in which instructions are executed in a Python program. Python executes code line by line from top to bottom, but this flow can be changed using control statements.

Types of Flow of Control
1. Sequential Flow
Default flow in Python
Statements are executed one after another in order

```
a = 10
b = 20
c = a + b
print(c)
```

---

2. Selection (Decision Making)
Used to make decisions based on conditions
Uses if, if-else, if-elif-else

```
age = 18

if age >= 18:
   print("Adult")
else:
   print("Minor")
```

---

3. Iteration (Looping)
Used to repeat a block of code multiple times
Uses for and while loops

```
for i in range(3):
   print(i)
```

Conditional Statements in Python

Conditional statements are used in Python to make decisions based on conditions. They allow a program to execute different blocks of code depending on whether a condition is True or False.

Types of Conditional Statements
1. if statement
Executes a block of code only if the condition is True.

```
x = 10

if x > 5:
   print("x is greater than 5")
```

---

2. if-else statement
Executes one block if condition is True, otherwise another block.

```
age = 16
```

```
if age >= 18:
   print("Adult")
else:
   print("Minor")
```

---

3. if-elif-else statement
Used when multiple conditions are checked.

```
marks = 75

if marks >= 90:
   print("A Grade")
elif marks >= 60:
   print("B Grade")
else:
   print("C Grade")
```

Nested if in Python
A nested if is an if statement inside another if statement. It is used when we need to check a condition only after another condition becomes true.

Syntax

```
if condition1:
   if condition2:
      # code block
```

Example

```
x = 10

if x > 5:
   print("x is greater than 5")

   if x > 8:
      print("x is also greater than 8")
```

LOOPS IN PYTHON

Loops in Python are used to execute a block of code repeatedly as long as a condition is true or for a fixed number of times.

Types of Loops in Python
1. for loop
Used when the number of repetitions is known.

```
for i in range(5):
   print(i)
```

Output: 0 1 2 3 4

---

2. while loop
Used when repetition depends on a condition.

```
i = 1

while i <= 5:
   print(i)
   i += 1
```

Loop Control Statements

break
Stops the loop immediately.

```
for i in range(5):
   if i == 3:
      break
   print(i)
```

---

continue
Skips the current iteration.

```
for i in range(5):
   if i == 2:
      continue
   print(i)
```

STRINGS IN PYTHON

A **string** in Python is a sequence of characters used to represent text. It is written inside **single quotes (' ')**, **double quotes (" ")**, or **triple quotes (''' ''' or """ """)**.

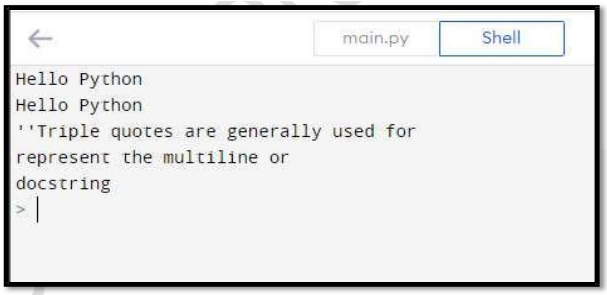
Key Features of Strings

- Strings are used to store text data

- They are **immutable** (cannot be changed after creation)
- Each character has an index (starting from 0)
- Can include letters, numbers, symbols, and spaces

CREATING STRING IN PYTHON

#Using single quotes str1 = 'Hello Python' print(str1)
#Using double quotes str2 = "Hello Python"
print(str2)
#Using triple quotes
str3 = '''''Triple quotes are generally used for
10. represent the multiline or
. docstring'''
. print(str3)



PYTHON STRING FORMATTING

Escape Sequences in Python

Escape sequences are special characters used in strings to represent actions like **new line, tab, quotes, etc.** They start with a **backslash **.

Common Escape Sequences

Escape Sequence	Meaning
\n	New line
\t	Tab space

Escape Sequence	Meaning
\\	Backslash
\'	Single quote
\"	Double quote
\r	Carriage return
\b	Backspace

FORMAT () METHOD

The **format()** method is the most flexible and useful method in formatting strings. The curly braces {} are used as the placeholder in the string and replaced by the **format()** method argument**.**

Python String Functions

String functions are built-in methods used to **manipulate and work with strings** in Python.

Common String Functions

1. lower()

Converts string to lowercase

```
text = "HELLO"
print(text.lower())   # hello
```

---

2. upper()

Converts string to uppercase

```
text = "hello"
print(text.upper())   # HELLO
```

---

3. strip()

Removes spaces from beginning and end

```
text = "  hello  "
print(text.strip())   # hello
```

---

4. replace()

Replaces one string with another

```
text = "Hello World"
print(text.replace("World", "Python"))
```

---

5. split()
Splits string into a list
```
text = "apple,banana,mango"
print(text.split(","))
```

---

6. join()
Joins list into a string
```
lst = ["apple", "banana"]
print(" ".join(lst))
```

---

7. find()
Finds position of a substring
```
text = "Hello"
print(text.find("e"))   # 1
```

---

8. count()
Counts occurrences of a substring
```
text = "banana"
print(text.count("a"))   # 3
```

---

9. startswith()
Checks if string starts with given value
```
text = "Python"
print(text.startswith("Py"))
```

---

10. endswith()
Checks if string ends with given value
```
text = "Python"
print(text.endswith("on"))
```

FUNCTIONS

A function is a block of reusable code that performs a specific task. It helps in breaking a large program into smaller, manageable parts.

Functions improve:

- Reusability (can be used multiple times)
- Modularity (divides program into parts)
- Code organization
- Avoids repetition

Python provides **built-in functions** like print() and range(), and also allows users to create their own functions called **user-defined functions**.

Types of Functions

1. **efine functions** - The user-defined functions are those define by the user to perform the specific task.
2. **Built-in functions** - The built-in functions are thosefunctions that are pre-defined in Python.

Advantage of Functions in Python

- There are the following advantages of Python functions.
- Using functions, we can avoid rewriting the same logic/code again and again in a program.
- We can call Python functions multiple times in a program and anywhere in a program.
- We can track a large Python program easily when it is divided into multiple functions.
- Reusability is the main achievement of Python functions.
- However, Function calling is alwaysoverhead in a Python program.

Creating a function

- Let's understand the syntax of functions definition.
- The def keyword, along with the function name is used to define the function.
- The identifier rule must follow the function name.
- A function accepts the parameter (argument), and they can be optional.
- The function block is started with the colon (:), and block statements must be at the same indentation.
- The return statementis used to return the value. A function can have only one return

FUNCTION CALLING

In Python, after the function is created, we can call it from
another function. A function must be defined before the function call; otherwise, the Python interpreter gives an
error. To call the function, use the function name followed by the parentheses.

Return statement

The return statement is used at the end of the function and returns the result of the function. It terminates the function execution and transfers the result where the function is called. The return statement cannot be used outside of the function

Arguments in function

The arguments are types of information which can be passed into the function. The arguments are specified in the parentheses. We can pass any number of arguments, but they must be separate them with a comma.

Consider the following example, which contains a function that accepts a string as the argument.

Example

```
1. #defining the function
2. def func (name):
3. print("Hi ",name)
4. #calling the function
5. func("yash")
```

types of argument

1. Required arguments
2. Keyword arguments
3. Default arguments
4. Variable-length arguments

Scope of variables

The scopes of the variables depend upon the location where the variable is being declared. The variable declared in one part of the program may not be accessible to the other parts.

In python, the variables are defined with the two types of scopes.

1. Global variables
2. Local variables

The variable defined outside any function is known to have a global scope, whereas the variable defined inside a function is known to have a local scope.